

Lista de atividades de Análise de Sobrevivência

Beatriz Lima Silveira

Questão 2

Enunciado

Implemente os modelos exponencial, gamma, log-normal e log-logístico no stan e ajuste os modelos implementados ao banco de dados sobrevivencia::isoladores.

Carregando pacotes

```
library(rstan)
```

Carregando pacotes exigidos: StanHeaders

rstan version 2.32.7 (Stan version 2.32.2)

For execution on a local, multicore CPU with excess RAM we recommend calling `options(mc.cores = parallel::detectCores())`.

To avoid recompilation of unchanged Stan programs, we recommend calling `rstan_options(auto_write = TRUE)`

For within-chain threading using ``reduce_sum()`` or ``map_rect()`` Stan functions, change ``threads_per_chain`` option:

```
rstan_options(threads_per_chain = 1)
```

```
library(sobrevivencia)
```

Carregando pacotes exigidos: survival

```
library(survival)
```

```
rstan_options(auto_write = TRUE)
```

```
options(mc.cores = parallel::detectCores())
```

Banco de dados

```
data(isoladores, package = "sobrevivencia")  
  
head(isoladores)
```

	tempo	evento
1	151	1
2	164	1
3	336	1
4	365	1
5	403	1
6	454	1

Preparação dos dados

```
N <- nrow(isoladores)  
  
stan_data <- list(  
  N = N,  
  t = isoladores$tempo,  
  d = isoladores$evento  
)
```

Modelo Exponencial

Assumimos:

$$T_i \sim \text{Exp}(\lambda)$$

A função de sobrevivência é dada por:

$$S(t) = e^{-\lambda t}$$

Código Stan

```
exp_stan <- '  
data {  
  int<lower=1> N;  
  vector<lower=0>[N] t;  
  array[N] int<lower=0,upper=1> d;  
}  
  
parameters {  
  real<lower=0> lambda;
```

```

}

model {

  lambda ~ gamma(0.01, 0.01);

  for (i in 1:N) {

    if (d[i] == 1)
      target += exponential_lpdf(t[i] | lambda);

    else
      target += exponential_lccdf(t[i] | lambda);

  }

}

'

writeLines(exp_stan, "stan/exponencial.stan" )

```

Compilação e ajuste

```

mod_exp <- stan_model(
  file = "stan/exponencial.stan"
)

fit_exp <- sampling(
  mod_exp,
  data = stan_data,
  iter = 4000,
  warmup = 2000,
  chains = 4,
  seed = 123
)

```

Resumo

```
print(fit_exp)
```

Inference for Stan model: anon_model.
 4 chains, each with iter=4000; warmup=2000; thin=1;
 post-warmup draws per chain=2000, total post-warmup draws=8000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
--	------	---------	----	------	-----	-----	-----	-------	-------	------

lambda	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	3052	1
lp__	-388.02	0.01	0.71	-390.11	-388.19	-387.75	-387.57	-387.52	3414	1	1

Samples were drawn using NUTS(diag_e) at Sun May 31 20:26:44 2026.
 For each parameter, n_eff is a crude measure of effective sample size,
 and Rhat is the potential scale reduction factor on split chains (at
 convergence, Rhat=1).

```
summary(fit_exp)$summary
```

	mean	se_mean	sd	2.5%	25%
lambda	4.965926e-04	1.343670e-06	7.422963e-05	3.590747e-04	4.438590e-04
lp__	-3.880205e+02	1.218204e-02	7.118015e-01	-3.901054e+02	-3.881862e+02
	50%	75%	97.5%	n_eff	Rhat
lambda	4.931728e-04	5.447986e-04	6.510936e-04	3051.895	1.001205
lp__	-3.877482e+02	-3.875654e+02	-3.875150e+02	3414.111	1.000250

A média a posteriori do parâmetro de taxa foi igual a ($\hat{\lambda}$)=0.000497), com intervalo de credibilidade de 95% dado por ((0.000359;0.000651)). Como no modelo exponencial o tempo médio de vida é dado por ($E(T) = 1/\lambda$), obteve-se um tempo médio estimado de aproximadamente 2012 horas.

Modelo Gamma

Assumimos:

$$T_i \sim \text{Gamma}(\alpha, \beta)$$

Código Stan

```
gamma_stan <- '
data {
  int<lower=1> N;
  vector<lower=0>[N] t;
  array[N] int<lower=0,upper=1> d;
}

parameters {
  real<lower=0> alpha;
  real<lower=0> beta;
}

model {
  alpha ~ gamma(0.01,0.01);
  beta ~ gamma(0.01,0.01);
}
```

```

for (i in 1:N) {

  if (d[i] == 1)
    target += gamma_lpdf(t[i] | alpha, beta);

  else
    target += gamma_lccdf(t[i] | alpha, beta);

}

}
'

writelines(gamma_stan, "stan/gamma.stan" )

```

Ajuste do modelo

```

mod_gamma <- stan_model( file = "stan/gamma.stan" )

fit_gamma <- sampling(
  mod_gamma,
  data = stan_data,
  iter = 4000,
  warmup = 2000,
  chains = 4,
  seed = 123
)

```

Resumo

```
print(fit_gamma)
```

Inference for Stan model: anon_model.
 4 chains, each with iter=4000; warmup=2000; thin=1;
 post-warmup draws per chain=2000, total post-warmup draws=8000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
alpha	1.54	0.01	0.31	1.01	1.32	1.52	1.74	2.20	1893	1
beta	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1895	1
lp__	-386.06	0.02	1.04	-388.75	-386.46	-385.75	-385.32	-385.04	2219	1

Samples were drawn using NUTS(diag_e) at Sun May 31 20:26:48 2026.
 For each parameter, n_eff is a crude measure of effective sample size,
 and Rhat is the potential scale reduction factor on split chains (at

```
convergence, Rhat=1).
```

```
summary(fit_gamma)$summary
```

	mean	se_mean	sd	2.5%	25%
alpha	1.543775e+00	7.023165e-03	0.3055451994	1.008607e+00	1.324662e+00
beta	8.290655e-04	4.901771e-06	0.0002133902	4.524807e-04	6.786191e-04
lp__	-3.860598e+02	2.199259e-02	1.0360380840	-3.887546e+02	-3.864584e+02
	50%	75%	97.5%	n_eff	Rhat
alpha	1.522375e+00	1.735837e+00	2.201697e+00	1892.715	1.004267
beta	8.085622e-04	9.640334e-04	1.287197e-03	1895.147	1.003516
lp__	-3.857528e+02	-3.853192e+02	-3.850399e+02	2219.212	1.000333

No modelo gama, a média a posteriori do parâmetro de forma foi igual a ($\hat{\alpha} = 1.544$), enquanto a média a posteriori do parâmetro de taxa foi ($\hat{\beta} = 0.000829$). Os respectivos intervalos de credibilidade de 95% foram ((1.009;2.202)) para (α) e ((0.000452;0.001287)) para (β). Como a média da distribuição gama é dada por $E(T) = \frac{\alpha}{\beta}$, obteve-se um tempo médio estimado de aproximadamente (1862) horas.

Modelo Log-Normal

Assumimos:

$$\log(T_i) \sim N(\mu, \sigma^2)$$

Código Stan

```
lnorm_stan <- '
data {
  int<lower=1> N;
  vector<lower=0>[N] t;
  array[N] int<lower=0,upper=1> d;
}

parameters {
  real mu;
  real<lower=0> sigma;
}

model {
  mu ~ normal(0,100);
  sigma ~ cauchy(0,5);

  for (i in 1:N) {
```

```

    if (d[i] == 1)
      target += lognormal_lpdf(t[i] | mu, sigma);

    else
      target += lognormal_lccdf(t[i] | mu, sigma);

  }

}
'

writelines(lnorm_stan, "stan/lognormal.stan" )

```

Compilação e ajuste

```

mod_lnorm <- stan_model( file = "stan/lognormal.stan" )

fit_lnorm <- sampling(
  mod_lnorm,
  data = stan_data,
  iter = 4000,
  warmup = 2000,
  chains = 4,
  seed = 123
)

```

Resumo

```
print(fit_lnorm)
```

Inference for Stan model: anon_model.
 4 chains, each with iter=4000; warmup=2000; thin=1;
 post-warmup draws per chain=2000, total post-warmup draws=8000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
mu	7.23	0.00	0.14	6.98	7.14	7.23	7.32	7.51	4524	1
sigma	0.99	0.00	0.12	0.80	0.91	0.98	1.06	1.26	4835	1
lp__	-383.79	0.02	1.02	-386.61	-384.17	-383.47	-383.07	-382.78	3356	1

Samples were drawn using NUTS(diag_e) at Sun May 31 20:26:51 2026.
 For each parameter, n_eff is a crude measure of effective sample size,
 and Rhat is the potential scale reduction factor on split chains (at
 convergence, Rhat=1).

```
summary(fit_lnorm)$summary
```

	mean	se_mean	sd	2.5%	25%	50%
mu	7.2346935	0.002024082	0.1361411	6.9751222	7.1425929	7.2316350
sigma	0.9931072	0.001683534	0.1170631	0.7985833	0.9095501	0.9806846
lp__	-383.7867964	0.017597020	1.0194828	-386.6130516	-384.1713590	-383.4722048

	75%	97.5%	n_eff	Rhat
mu	7.322626	7.510931	4523.999	1.000329
sigma	1.064353	1.257650	4834.998	1.000196
lp__	-383.065885	-382.784584	3356.460	1.000645

Para o modelo log-normal, as médias a posteriori dos parâmetros foram ($\hat{\mu} = 7.235$) e ($\hat{\sigma} = 0.993$), com intervalos de credibilidade de 95% iguais a (6.975;7.511) para (μ) e (0.799;1.258) para (σ).

Na parametrização log-normal, o tempo médio de vida é dado por $E(T) = \exp\left(\mu + \frac{\sigma^2}{2}\right)$.

Substituindo as estimativas obtidas, encontrou-se um tempo médio estimado de aproximadamente (2075) horas. Os diagnósticos de convergência indicaram estabilidade das cadeias simuladas, com valores de $\hat{R}$ próximos de (1).

Modelo Log-Logístico

A função de sobrevivência do modelo log-logístico é:

$$S(t) = \frac{1}{1 + (t/\alpha)^\beta}$$

Neste exercício implementamos manualmente a densidade e a função de sobrevivência do modelo log-logístico utilizando funções definidas pelo usuário no Stan.

Código Stan

```
llogis_stan <- '
functions {
  real my_loglogistic_lpdf(real t, real alpha, real beta) {
    return log(beta) - log(alpha) +
      (beta - 1) * (log(t) - log(alpha)) -
      2 * log1p(pow(t / alpha, beta));
  }
}
```

```

    real my_loglogistic_lccdf(real t, real alpha, real beta) {

        return -log1p(pow(t / alpha, beta));

    }

}

data {

    int<lower=1> N;
    vector<lower=0>[N] t;
    array[N] int<lower=0,upper=1> d;

}

parameters {

    real<lower=0> alpha;
    real<lower=0> beta;

}

model {

    alpha ~ gamma(0.01,0.01);
    beta ~ gamma(0.01,0.01);

    for (i in 1:N) {

        if (d[i] == 1)
            target += my_loglogistic_lpdf(t[i] | alpha, beta);

        else
            target += my_loglogistic_lccdf(t[i] | alpha, beta);

    }

}

'

writeLines(llogis_stan, "stan/loglogistico.stan" )

```

Compilação e ajuste

```
mod_llogis <- stan_model(file = "stan/loglogistico.stan" )

fit_llogis <- sampling(
  mod_llogis,
  data = stan_data,
  iter = 4000,
  warmup = 2000,
  chains = 4,
  seed = 123
)
```

Resumo

```
print(fit_llogis)
```

Inference for Stan model: anon_model.
4 chains, each with iter=4000; warmup=2000; thin=1;
post-warmup draws per chain=2000, total post-warmup draws=8000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
alpha	1122.69	1.94	137.93	863.60	1027.75	1116.92	1213.87	1400.94	5066	1
beta	1.73	0.00	0.22	1.32	1.58	1.73	1.87	2.18	5725	1
lp__	-396.48	0.02	1.03	-399.27	-396.88	-396.16	-395.74	-395.48	2922	1

Samples were drawn using NUTS(diag_e) at Sun May 31 20:26:53 2026.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).

```
summary(fit_llogis)$summary
```

	mean	se_mean	sd	2.5%	25%	50%
alpha	1122.691583	1.937814972	137.9252605	863.600131	1027.746657	1116.918177
beta	1.731854	0.002885231	0.2183094	1.319044	1.582981	1.731374
lp__	-396.480608	0.019010577	1.0275498	-399.273866	-396.877786	-396.155646
	75%	97.5%	n_eff	Rhat		
alpha	1213.866298	1400.944564	5065.975	1.000527		
beta	1.871359	2.177826	5725.107	1.000338		
lp__	-395.738969	-395.482135	2921.562	1.002363		

No modelo log-logístico, as médias a posteriori dos parâmetros foram ($\hat{\alpha} = 1122.692$) e ($\hat{\beta} = 1.732$). Os intervalos de credibilidade de 95% foram ((863.600;1400.945)) para (α) e ((1.319;2.178)) para (β).

Como (> 1), a média da distribuição log-logística existe e é dada por $E(T) =$

$\alpha \frac{\pi/\beta}{\sin(\pi/\beta)}$. Utilizando as estimativas a posteriori, obteve-se um tempo médio estimado de aproximadamente 2034 horas

Questão 3

i Enunciado

Implemente no Stan as versões frequentista e Bayesiana do modelo AFT com distribuição basal Weibull. Compare as estimativas dos parâmetros.

Para implementar as versões frequentista e Bayesiana do modelo AFT Weibull e comparar os resultados, será gerados dados simulados com distribuição Weibull.

```
library(rstan)
library(survstan)
library(sobrevivencia)
library(survival)

rstan_options(auto_write = TRUE)
options(mc.cores = parallel::detectCores())
```

Neste estudo geramos dados de um modelo Weibull com $\alpha = 1.5$ e $\sigma = 1000$, além de censura aleatória a direita.

```
set.seed(123)

n <- 100

alpha_real <- 1.5
sigma_real <- 1000

# tempos reais de falha
T_real <- rweibull(
  n,
  shape = alpha_real,
  scale = sigma_real
)

# tempos de censura
C <- runif(n, min = 500, max = 2500)

# tempos observados
tempo <- pmin(T_real, C)

# indicador de falha
```

```
evento <- as.numeric(T_real <= C)

dados <- data.frame(
  tempo = tempo,
  evento = evento
)

head(dados)
```

```
      tempo evento
1 1158.0832      1
2  383.9074      1
3  928.0890      1
4  249.2132      1
5  155.6003      1
6 2120.9318      1
```

Ajuste frequentista com Survstan

```
fit_freq <- aftreg(
  Surv(tempo, evento) ~ 1,
  data = dados,
  baseline = "weibull"
)
```

Ajuste do Modelo Weibull com Stan

```
stan_data <- list(
  N = nrow(dados),
  t = dados$tempo,
  d = dados$evento
)

weibull_stan <- '
data {
  int<lower=1> N;
  vector<lower=0>[N] t;
  array[N] int<lower=0,upper=1> d;
  int<lower=0,upper=1> is_bayes;
}

parameters {
```

```

    real<lower=0> alpha;
    real<lower=0> sigma;

}

model {

    if(is_bayes == 1){

        alpha ~ gamma(0.01, 0.01);
        sigma ~ gamma(0.01, 0.01);

    }

    for(i in 1:N){

        if(d[i] == 1)
            target += weibull_lpdf(t[i] | alpha, sigma);

        else
            target += weibull_lccdf(t[i] | alpha, sigma);

    }

}

,

writeLines(
    weibull_stan,
    "stan/weibull_simulacao.stan"
)

```

Compilação do modelo

```

mod_weibull <- stan_model(
    file = "stan/weibull_simulacao.stan"
)

```

No modelo implementado, a variável `is_bayes` foi utilizada para controlar o tipo de ajuste realizado:

- `is_bayes = 0`: ajuste frequentista;
- `is_bayes = 1`: ajuste Bayesiano.

Quando `is_bayes = 1`, o modelo inclui prioris para os parâmetros:

$$\alpha \sim \text{Gamma}(0.01, 0.01)$$

e

$$\sigma \sim \text{Gamma}(0.01, 0.01)$$

Caso contrário, o modelo utiliza apenas a abordagem frequentista.

Ajuste frequentista no Stan

```
stan_data_freq <- list(
  N = nrow(dados),
  t = dados$tempo,
  d = dados$evento,
  is_bayes = 0
)

fit_freq_stan <- optimizing(
  mod_weibull,
  data = stan_data_freq,
  hessian = TRUE
)
```

Estimativas frequentistas

```
fit_freq_stan$par

      alpha      sigma
1.519851 1034.590226
```

Matriz Hessiana

```
fit_freq_stan$hessian

      alpha      sigma
alpha -124.89948   17.25481
sigma  17.25481 -187.10515
```

Matriz de variâncias e covariâncias

```
V <- solve(-fit_freq_stan$hessian)

V
```

```

              alpha      sigma
alpha 0.0081097578 0.0007478806
sigma 0.0007478806 0.0054135578

```

Intervalos de confiança

```

estimativas <- fit_freq_stan$par

ep <- sqrt(diag(V))

li <- estimativas - qnorm(0.975)*ep
ls <- estimativas + qnorm(0.975)*ep

cbind(
  estimativas,
  ep,
  li,
  ls
)

```

```

      estimativas      ep      li      ls
alpha    1.519851 0.09005419    1.343348    1.696354
sigma 1034.590226 0.07357688 1034.446018 1034.734434

```

As estimativas frequentistas foram obtidas por máxima verossimilhança utilizando a função `optimizing()` do pacote `rstan`.

Ajuste Bayesiano

```

stan_data_bayes <- list(
  N = nrow(dados),
  t = dados$tempo,
  d = dados$evento,
  is_bayes = 1
)

fit_bayes <- sampling(
  mod_weibull,
  data = stan_data_bayes,
  iter = 4000,
  warmup = 2000,
  chains = 4,
  seed = 123
)

```

Resumo Bayesiano

```
print(fit_bayes)
```

Inference for Stan model: anon_model.

4 chains, each with iter=4000; warmup=2000; thin=1;

post-warmup draws per chain=2000, total post-warmup draws=8000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
alpha	1.49	0.00	0.13	1.23	1.40	1.49	1.58	1.77	5106	1
sigma	984.68	0.96	71.81	851.84	935.61	983.22	1032.28	1129.06	5644	1
lp__	-644.22	0.02	1.02	-647.03	-644.61	-643.90	-643.50	-643.24	3256	1

Samples were drawn using NUTS(diag_e) at Sun May 31 20:26:56 2026.

For each parameter, n_eff is a crude measure of effective sample size, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat=1).

```
summary(fit_bayes)$summary
```

	mean	se_mean	sd	2.5%	25%	50%
alpha	1.487973	0.001880448	0.1343711	1.234631	1.395178	1.486233
sigma	984.676351	0.955849077	71.8082940	851.838206	935.607251	983.220343
lp__	-644.224489	0.017918058	1.0224661	-647.027186	-644.611972	-643.903973

	75%	97.5%	n_eff	Rhat
alpha	1.575478	1.767552	5106.098	1.0008274
sigma	1032.279759	1129.055301	5643.786	1.0002545
lp__	-643.501108	-643.236227	3256.237	0.9998305

Comparação entre os modelos

Valores reais utilizados na simulação

```
c(  
  alpha_real = alpha_real,  
  sigma_real = sigma_real  
)
```

alpha_real	sigma_real
1.5	1000.0

Estimativas frequentistas via survstan

```
fit_freq$estimates
```

alpha	gamma
1.519852	1034.590587

Estimativas frequentistas via Stan

```
fit_freq_stan$par
```

	alpha	sigma
	1.519851	1034.590226

Estimativas Bayesianas

```
summary(fit_bayes)$summary
```

	mean	se_mean	sd	2.5%	25%	50%
alpha	1.487973	0.001880448	0.1343711	1.234631	1.395178	1.486233
sigma	984.676351	0.955849077	71.8082940	851.838206	935.607251	983.220343
lp__	-644.224489	0.017918058	1.0224661	-647.027186	-644.611972	-643.903973
	75%	97.5%	n_eff	Rhat		
alpha	1.575478	1.767552	5106.098	1.0008274		
sigma	1032.279759	1129.055301	5643.786	1.0002545		
lp__	-643.501108	-643.236227	3256.237	0.9998305		

Os modelos frequentistas ajustados via `survstan` e via `Stan` apresentaram estimativas bastante semelhantes, como esperado, uma vez que ambos utilizam máxima verossimilhança.

As estimativas Bayesianas também ficaram próximas dos valores reais utilizados na simulação, indicando que as prioris pouco informativas exerceram pequena influência sobre os resultados finais.